P.PORTO

ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

Relatório

Base de Dados

2025/2026

Relatório elaborado por:

Grupo 112

8240344 Mariana Silva Barreto

8240227 Ricardo Filipe Fernandes Lopes

8240268 David Gabriel Ventura Fernandes

Curso de:

Licenciatura em Engenharia Informática

Docentes:

Vasco Santos (vsantos@eu.ipp.pt)

Bruno Oliveira (bmo@estg.ipp.pt)

Felgueiras, 29 de maio de 2026

Resumo

Este relatório descreve a criação de uma base de dados para gerir a formação de um clube desportivo. O objetivo do sistema é centralizar e automatizar tarefas essenciais, como a gestão de atletas e encarregados de educação, alocação de treinadores, agendamento de treinos, controlo de assiduidade, pagamento de quotas e emissão de alertas

A metodologia adotada percorreu rigorosamente as fases de levantamento de requisitos, modelação conceptual, transição para o modelo lógico — com validação estrita até à Terceira Forma Normal (3FN) — e, por fim, o desenho físico. O resultado culmina num esquema relacional implementado em *Microsoft SQL Server*, complementado por mecanismos de *Triggers*, *Views* e *Stored Procedures* que asseguram o cumprimento das regras de negócio.

De forma a garantir a máxima integridade referencial e eficiência computacional, o projeto destaca-se por decisões estruturais otimizadas, tais como a centralização associativa através da entidade `Escalao_Epoca` e a adoção do padrão *Single Table Inheritance* para a gestão unificada de alertas.

Índice

Resumo **1**

1 Introdução **6**

1.1 Contextualização do Problema 6

1.2 Apresentação do Caso de Estudo 6

1.3 Motivação e Objetivos 7

1.4 Estrutura do Relatório 7

2 Recolha e Análise de Dados **8**

2.1 Regras de Negócio 8

3 Desenho Conceptual da Base de Dados **10**

3.1 Identificar Tipos de Entidades 10

3.2 Identificar e Associar Atributos com Entidades ou Relacionamentos 11

3.3 Determinar Domínios de Atributos 11

3.4 Determinar Chaves Primárias, Candidatas e Alternativas 13

3.5 Modelo Conceptual (Diagrama ER) 15

3.5.1 Decisões tomadas pelo Grupo 16

3.6 Verificar a Redundância do Modelo 17

3.7 Identificar os Caminhos que Suportam as Transações 17

4 Desenho Lógico da Base de Dados **19**

4.1 Obtenção do Modelo Lógico 19

4.1.1 Mapeamento de Entidades Fortes 19

4.1.2 Mapeamento de Relacionamentos 1:* (Um-para-Muitos) . 19

4.1.3 Mapeamento de Relacionamentos Muitos-para-Muitos (*:*) 20

4.1.4 Mapeamento de Especialização (Herança de Alertas) . . . 20
4.1.5 Justificação das Decisões Estruturais do Grupo 20
4.2 Desenho do Modelo Lógico 22
4.2.1 Especificação do Desenho do Modelo Lógico 23
4.3 Normalização 25
4.3.1 Primeira Forma Normal (1FN) 25
4.3.2 Segunda Forma Normal (2FN) 26
4.3.3 Terceira Forma Normal (3FN) 27

5 Desenho Físico **28**
5.1 Criação de Tabelas e Restrições de Integridade 28
5.1.1 Estrutura Base e Chaves Primárias 28
5.1.2 Integridade Referencial (Chaves Estrangeiras) 29
5.1.3 Restrições de Domínio e Regras de Negócio 29
5.1.4 Exemplo de Implementação: Tabela de Inscrição 30
5.2 Views 30
5.2.1 Exemplo de Implementação: Listagem de Jogadores por Escalão 31
5.3 Stored Procedures 32
5.3.1 Exemplo de Implementação: Registo de Inscrição de Jogador 33
5.4 Triggers 35
5.4.1 Exemplo de Implementação: Validação de Menores de Idade 35

6 Conclusões **37**

Bibliografia **38**

Anexos **40**

Índice de Figuras

3.1 Diagrama Conceptual da Base de Dados 15

4.1 Diagrama do Modelo Lógico 22

4.2 Exemplo de Aplicação da 1FN (Remoção de Grupos Repetitivos) 25

4.3 Exemplo de Aplicação da 2FN (Remoção de Dependências Parciais) 26

4.4 Exemplo de Aplicação da 3FN (Remoção de Dependências Transitivas) 27

6.1 Diagrama do Modelo Lógico 40

Índice de Tabelas

3.1 Identificação das Entidades 10

3.2 Identificação dos Relacionamentos Conceptuais 11

3.3 Decisões do Grupo 16

Introdução

1.1 Contextualização do Problema

Um clube desportivo de futebol organiza a sua atividade de formação em vários escalões, cada um com treinadores, jogadores inscritos e calendários próprios. A gestão desta atividade envolve uma elevada quantidade de informação heterogénea que, sem um sistema adequado, tende a dispersar-se em folhas de cálculo, tornando difícil a consulta e o controlo eficaz.
A ausência de um sistema centralizado dificulta tarefas como verificar quotas em atraso ou acompanhar a assiduidade dos jogadores, criando ineficiências administrativas.

1.2 Apresentação do Caso de Estudo

O caso de estudo descreve um clube desportivo que pretende implementar uma base de dados relacional para gerir:

- Escalões e respetivas equipas técnicas;
- Jogadores inscritos, encarregados de educação e estado da inscrição;
- Calendário de treinos e presenças;
- Pagamentos de quotas;
- Sistema de alertas para situações críticas (assiduidade baixa e quotas em atraso).

1.3 Motivação e Objetivos

A principal motivação para a realização deste trabalho é a aplicação prática dos conceitos de Modelação de Bases de Dados num cenário real de gestão desportiva. O objetivo central é desenvolver uma Base de Dados Relacional eficiente, passando pelas fases de Desenho Conceptual, Lógico e Físico.

Como objetivos específicos, destacam-se:

- Garantir a integridade dos dados através da normalização (3FN).
- Automatizar regras de negócio complexas (como alertas, sobreposição de treinos e gestão de quotas) através de *Triggers* e *Stored Procedures*.
- Assegurar a preservação do histórico do clube, utilizando a eliminação lógica (marcação de inativos) em vez de apagar os registos fisicamente.

1.4 Estrutura do Relatório

O presente relatório está organizado em várias secções: o Capítulo 2 aborda a recolha e análise de requisitos, definindo as **regras de negócio**. O Capítulo 3 apresenta o **Desenho Conceptual** através do Modelo Entidade-Relacionamento. O Capítulo 4 detalha a passagem para o **Desenho Lógico** e o processo de **Normalização**. O Capítulo 5 expõe o **Desenho Físico** e a implementação no *SQL Server*. Por fim, são apresentadas as Conclusões, a Bibliografia e os respetivos Anexos.

Recolha e Análise de Dados

2.1 Regras de Negócio

A análise do enunciado e o enriquecimento do caso de estudo permitiram identificar as seguintes regras de negócio:

RN01 – Estrutura de Escalões e Equipas Técnicas

- Cada escalão é definido por um intervalo de idades (ex.: Benjamins: 8-9 anos);
- Cada escalão tem uma equipa técnica por época, constituída obrigatoriamente por um Treinador Principal e, opcionalmente, por Treinadores Adjuntos e um Treinador de Guarda-Redes;
- Um treinador pode participar em diferentes escalões e épocas, com funções distintas.

RN02 – Inscrições de Jogadores

- O estado inicial de qualquer inscrição é automaticamente definido como 'Pendente';
- Uma inscrição só transita para 'Ativa' após validação pela direção;
- Um jogador não pode ter duas inscrições ativas na mesma época desportiva;
- A idade do jogador deve ser compatível com o intervalo de idades do escalão em que é inscrito;
- Antes de validar uma nova inscrição, é necessário verificar que não existem quotas em atraso de épocas anteriores;
- Jogadores menores de 18 anos devem ter um Encarregado de Educação registado.

RN03 – Inscrições Federativas

- Uma inscrição federativa está associada a uma inscrição no clube;
- Quando a data de validade da inscrição federativa é ultrapassada, é gerado automaticamente um alerta;
- O número federado é único em todo o sistema.

RN04 – Treinos e Presença

- Ao registar um treino como realizado, são criados automaticamente registos de presença (estado 'Ausente') para todos os jogadores com inscrição ativa nesse escalão e época;
- Não é possível registar assiduidade de treinos com data futura;
- Ao marcar a presença de um jogador, a taxa de assiduidade acumulada é recalculada automaticamente;
- Se a taxa de assiduidade descer abaixo de 50%, é gerado um alerta.

RN05 – Quota e Pagamentos

- Ao validar uma inscrição, é gerada automaticamente a primeira quota mensal;
- As quotas têm um valor fixo por inscrição (definido pela direção) e uma data de vencimento no fim do mês de referência;
- Uma quota pode ter no máximo um pagamento associado;
- Quotas vencidas sem pagamento registado são consideradas 'Em Atraso'.

RN06 – Alertas

- Os alertas são categorizados por Tipo de Alerta (Assiduidade baixa, Quota em atraso, etc.);
- Um alerta ativo do mesmo tipo não é duplicado para o mesmo jogador;
- Os alertas podem ser marcados como 'Resolvidos'.

Desenho Conceptual da Base de Dados

3.1 Identificar Tipos de Entidades

Com base na análise de requisitos do clube desportivo, foram identificados os seguintes tipos de entidades:

Entidade	**Descrição**
`EncarregadoEducacao` (Forte)	Responsável legal dos atletas menores.
`Jogador` (Forte)	Entidade central que representa os atletas de formação no clube.
`EpocaDesportiva` (Forte)	Define o âmbito temporal das atividades (ex.: 2025/2026).
`Escalao` (Forte)	Define as divisões desportivas por idades (ex.: Sub-15).
`Treinador` (Forte)	Representa os profissionais técnicos do clube.
`Treino` (Forte)	Regista as sessões de treino planeadas e realizadas.
`Alerta` (Forte)	Entidade genérica de base para notificações automáticas.
`Inscricao` (Associativa)	Registo da participação de um jogador num escalão durante uma época desportiva.
`Quota` (Associativa)	Mensalidades geradas a partir de uma inscrição ativa.
`Pagamento` (Fraca)	Registo de pagamento de uma Quota.
`EquipaTecnica` (Associativa)	Entidade associativa que representa a composição técnica de um escalão numa determinada época. Resolve a relação ternária entre Treinadores, Escalões e Épocas Desportivas.
`Presenca` (Associativa)	Mapeia a assiduidade dos Jogadores em cada Treino.
`AlertaPagamento/Presenca`	Subentidades que dependem da entidade `Alerta`.

Tabela 3.1: Identificação das Entidades

3.2 Identificar e Associar Atributos com Entidades ou Relacionamentos

A tabela seguinte descreve os relacionamentos conceptuais entre as entidades identificadas, adotando a notação de multiplicidade proposta por Connolly e Begg:

Entidades	**Cardinalidade**	**Descrição**
Encarregado – Jogador	**1:***	Um encarregado é responsável por um ou vários jogadores (1..∗); cada jogador tem um único encarregado (1..1).
Jogador – Inscricao	**1:***	Um jogador tem várias inscrições ao longo dos anos (1..∗); cada inscrição pertence a um só jogador (1..1).
Inscricao – Quotas	**1:***	Uma inscrição gera múltiplas quotas (1..∗); cada quota é de uma inscrição única (1..1).
Quotas – Pagamento	**1:1**	Uma quota pode não estar paga (0..1); um pagamento pertence obrigatoriamente a uma quota (1..1).
Escalao – Treino	**1:***	Um escalão possui múltiplos treinos (0..∗); um treino pertence obrigatoriamente a um escalão (1..1).
Jogador – Treino	***:***	Relacionamento resolvido através da entidade associativa `Presenca`.
Treinador–Escalao–Epoca	***:*:***	Relacionamento ternário resolvido através da entidade associativa `Equipa_Tecnica`.

Tabela 3.2: Identificação dos Relacionamentos Conceptuais

3.3 Determinar Domínios de Atributos

A definição dos domínios estabelece os tipos de dados e as restrições de valores que cada atributo pode assumir no sistema, garantindo a integridade do domínio

desde a fase conceptual. Os domínios definidos para o modelo são:

- **Domínio IDs (Identificadores):** Números inteiros, positivos, gerados sequencialmente em base automática para atuarem como chaves primárias.
- **Domínio Nomes:** Conjunto de caracteres alfanuméricas (até 100 caracteres), obrigatórias (*NOT NULL*), sem numeração intercalada.
- **Domínio Contactos:** Conjunto de caracteres numéricos com formatação estrita de 9 dígitos para telemóveis nacionais.
- **Domínio Emails:** Conjunto de caracteres textuais com validação obrigatória de máscara padrão (`exemplo@gmail.com`).
- **Domínio de Datas e Horas:** Subdividido em dois tipos temporais distintos para acomodar os requisitos do sistema:
  - **Domínio de Data:** Utilizado para atributos onde apenas o dia, mês e ano são relevantes, como é o caso da `dataNasc` da entidade Jogador, da `data` da entidade Treino, ou da `dataInscricao`.
  - **Domínio de Hora:** Aplicado especificamente aos atributos `horaInicio` e `horaFim` da entidade Treino. Este domínio é logicamente limitado ao horário de funcionamento operacional do clube (08:00 às 22:30) e assume um papel indispensável na futura validação de sobreposições de agendamentos.
- **Domínio Valores Financeiros:** Valores decimais com precisão monetária de duas casas decimais, obrigatoriamente superiores a zero (`valor > 0`).
- **Domínio Estados de Inscrição:** Valores textuais discretos limitados ao conjunto: {`'Pendente'`, `'Ativa'`, `'Suspensa'`, `'Cancelada'`}.
- **Domínio Estados de Presença:** Valores textuais discretos limitados ao conjunto: {`'Presente'`, `'Falta Justificada'`, `'Falta Injustificada'`}.
- **Domínio Funções Técnicas:** Valores textuais discretos limitados ao conjunto: {`'Treinador Principal'`, `'Adjunto'`, `'Treinador GR'`, `'Preparador Físico'`}.
- **Domínio Métodos de Pagamento:** Valores textuais discretos limitados ao conjunto: {`'MB Way'`, `'Multibanco'`, `'Transferência'`, `'Numerário'`}.
- **Domínio de Estados Lógicos (Booleanos):** Valores abstratos de dupla condição (Sim/Não), aplicados unicamente ao atributo `realizado` na

entidade `Treino`, que determina se a sessão de treino já foi executada ou se continua apenas agendada.

3.4 Determinar Chaves Primárias, Candidatas e Alternativas

Seguindo as diretrizes metodológicas, foram analisados todos os candidatos a chaves para cada entidade conceptual, selecionando a Chave Primária (PK) e isolando as Chaves Alternativas (AK):

1. **Encarregado_Educacao:**
   - *Chaves Candidatas:* {`id_enc_educacao`}, {`contacto`}, {`email`}
   - *Chave Primária (PK):* `id_enc_educacao`
   - *Chaves Alternativas (AK):* `contacto` (AK1), `email` (AK2)
2. **Jogador:**
   - *Chaves Candidatas:* {`id_jogador`}, {`nome, data_nascimento`}
   - *Chave Primária (PK):* `id_jogador`
   - *Chaves Alternativas (AK):* `nome, data_nascimento` (AK1)
3. **Inscricao:**
   - *Chaves Candidatas:* {`id_inscricao`}, {`nr_federado`}, {`id_jogador, id_escalao_epoca`}
   - *Chave Primária (PK):* `id_inscricao`
   - *Chaves Alternativas (AK):* `nr_federado` (AK1), `id_jogador, id_escalao_epoca` (AK2)
4. **Quotas:**
   - *Chaves Candidatas:* {`id_quota`}, {`id_inscricao, data_vencimento`}
   - *Chave Primária (PK):* `id_quota`
   - *Chaves Alternativas (AK):* `id_inscricao, data_vencimento` (AK1)
5. **Pagamento:**
   - *Chaves Candidatas:* {`id_pagamento`}, {`id_quota`}
   - *Chave Primária (PK):* `id_pagamento`
   - *Chaves Alternativas (AK):* `id_quota` (AK1 - garante a relação estrita de 1:1)

6. **Treino:**
   - *Chaves Candidatas:* {`id_treino`}, {`data_hora, local`}
   - *Chave Primária (PK):* `id_treino`
   - *Chaves Alternativas (AK):* `data_hora, local` (AK1)
7. **Presenca:**
   - *Chaves Candidatas:* {`id_jogador, id_treino`}
   - *Chave Primária (PK):* Composta por (`id_jogador, id_treino`)
   - *Chaves Alternativas (AK):* Não existem.
8. **Treinador:**
   - *Chaves Candidatas:* {`id_treinador`}, {`contacto`}
   - *Chave Primária (PK):* `id_treinador`
   - *Chaves Alternativas (AK):* `contacto` (AK1)
9. **Equipa_Tecnica:**
   - *Chaves Candidatas:* {`id_treinador, id_escalao_epoca`}
   - *Chave Primária (PK):* Composta por (`id_treinador, id_escalao_epoca`)
   - *Chaves Alternativas (AK):* Não existem.

3.5 Modelo Conceptual (Diagrama ER)

O Diagrama Entidade-Relacionamento (ER) foi elaborado com base na metodologia de Connolly e Begg [0] e encontra-se representado na figura seguinte.

No nosso modelo, destacamos a representação gráfica da Herança (Especialização) nos Alertas e o rigor das cardinalidades. O diagrama utiliza a notação UML para modelação de dados.

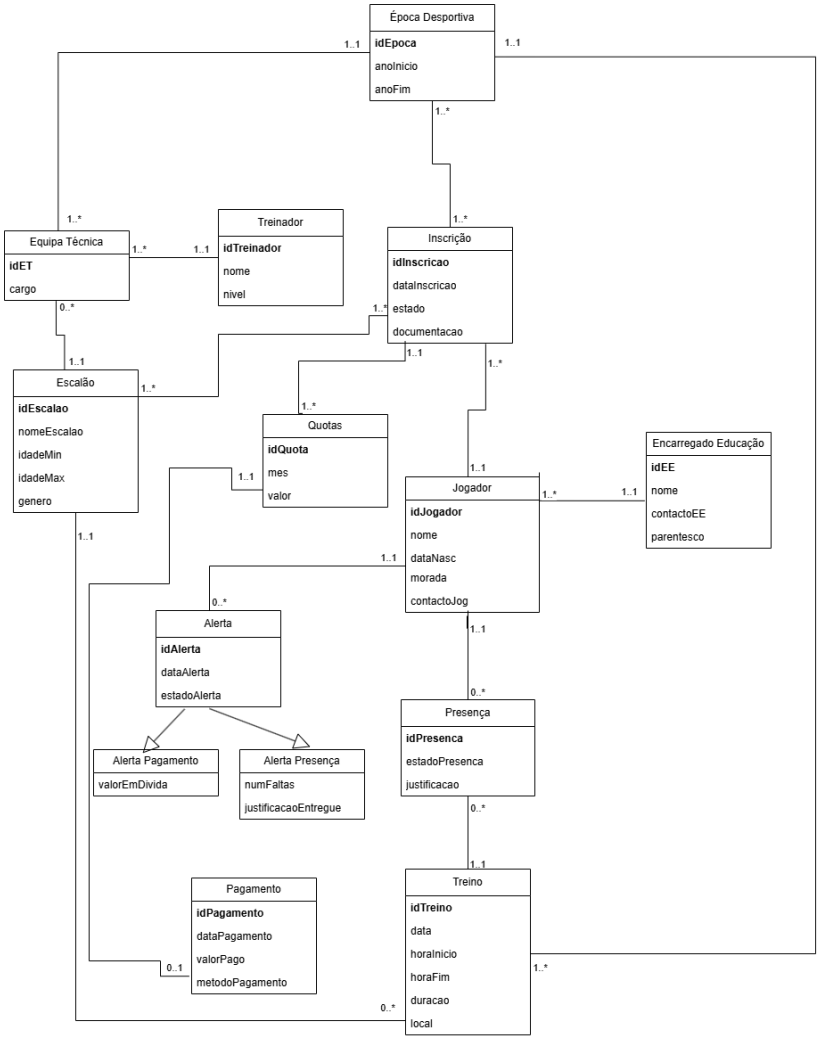


Figura 3.1: Diagrama Conceptual da Base de Dados

3.5.1 Decisões tomadas pelo Grupo

Nesta secção, consolidam-se as opções estratégicas adotadas pelo grupo:

Decisão Estrutural	**Justificação**
Atributo `funcao` na entidade `Equipa_Tecnica`	Permite modelar o relacionamento muitos-para-muitos, diferenciando o papel funcional (e.g., Principal, Adjunto) de cada técnico dentro da mesma equipa sem criar redundância de tabelas.
Atributo `realizado` na entidade `Treino`	Facilita a distinção lógica entre o agendamento futuro e a concretização do treino. Funciona como condição para a geração e inserção automática dos registos de presença.
Atributo `metodo_pagamento` na entidade `Pagamento`	Enriquece o modelo para efeitos de auditoria financeira, permitindo rastrear a origem da receita e facilitar futuras integrações com plataformas externas (MBWay, Transferência, Numerário).
Atributo `taxa_assiduidade` na entidade `Presenca`	Atua como um campo calculado. Esta decisão evita a sobrecarga de sistema provocada por cálculos intensivos sobre todo o histórico de presenças sempre que se consulta o perfil de um jogador.
Atributo `relacao` na entidade `Encarregado_Educacao`	Garante flexibilidade no registo da figura parental ou legal (Pai, Mãe, Avô, Instituição), essencial para a correta identificação em comunicações e emergências.
Remoção da entidade "Inscrição Federativa"	O grupo decidiu colapsar os dados federativos (`nr_federado` e respetiva data de validade) diretamente na entidade `Inscricao`, assumindo uma relação 1:1, o que reduz substancialmente a necessidade de operações de *JOIN* dispendiosas.
Estratégia de "Tabela Única"na Herança de Alertas	Em vez de criar tabelas separadas para `AlertaPagamento` e `AlertaPresenca`, o grupo consolidou tudo na entidade `Alerta`, usando o campo `tipo` como discriminador. Isto simplifica enormemente o esquema lógico e acelera as pesquisas, à custa de alguns valores nulos controlados.

Tabela 3.3: Decisões do Grupo

3.6 Verificar a Redundância do Modelo

Para garantir a eficiência da base de dados, o grupo realizou uma validação crítica de caminhos cíclicos e redundâncias geométricas no diagrama conceitual.

- **Análise de Jogador - Inscrição - Escalão_Época:** Existe um caminho fechado aparente entre o atleta e a sua equipa. Contudo, esta relação não é redundante, pois a entidade associativa `Escalao_Epoca` isola a componente temporal das épocas desportivas, enquanto a `Inscricao` valida a participação individual do jogador. Sem ambas, seria impossível rastrear o histórico de um atleta que jogue múltiplas épocas no mesmo clube.
- **Remoção do Atributo Valor no Pagamento:** Inicialmente, ponderou-se incluir o campo `valor_pago` na entidade `Pagamento`. No entanto, aplicando as regras de eliminação de redundâncias, o grupo deliberou que, como o clube opera sob uma política estrita de liquidação integral, o valor pago é idêntico ao valor faturado na `Quota`. A sua remoção evita desoneração de armazenamento desnecessária.
- **Otimização das Subentidades de Alerta:** A especialização da entidade `Alerta` unifica os atributos comuns (mensagens, datas) na entidade pai, eliminando a duplicação de estruturas e simplificando caminhos de leitura concêntricos.

3.7 Identificar os Caminhos que Suportam as Transações

Esta validação metodológica garante que o modelo conceptual desenhado possui todos os relacionamentos e caminhos necessários para executar as operações e transações vitais do dia-a-dia do clube desportivo. Foram validados os seguintes mapeamentos transacionais:

- **Transação 1 - Renovação/Inscrição de Atleta:** O caminho inicia-se na verificação do `Jogador`. Se for menor, percorre a associação para `Encarregado_Educacao`. Prossegue para a criação da `Inscricao`, que

por sua vez se conecta a uma ocorrência de `Escalao_Epoca`, garantindo o enquadramento na época e faixa etária corretas.

- **Transação 2 - Faturação e Cobrança de Mensalidades:** O fluxo transacional mapeia-se de `Inscricao` → `Quota`. No momento do recebimento, a transação estende-se de `Quota` → `Pagamento`, fechando o fluxo financeiro de forma linear e isolada.
- **Transação T3 - Controlo Desportivo (Agendamento e Presenças):** Permite ao coordenador desportivo selecionar um determinado `Escalão` integrado numa `Época Desportiva` específica para efetuar o agendamento de um `Treino`. A partir desta ação, o fluxo conceptual navega através do relacionamento de assiduidade (`Presença`) para associar o treino a todos os membros da entidade `Jogador` que se encontram vinculados àquele escalão, preparando o sistema para o registo e controlo de faltas ou presenças.
- **Transação 4 - Consulta de Histórico:** Permite à direção consultar o registo histórico, desportivo e financeiro completo de atletas ou treinadores que já terminaram o seu vínculo prático com o clube. O fluxo inicia-se em `Jogador` ou `Treinador` e navega através das tabelas associativas para aceder a `Inscricao`, `Presenca`, `Quota` e `Equipa_Tecnica`. Esta necessidade do negócio exige que o modelo garanta a permanência destes caminhos relacionais, de modo a que o cessar de atividade de um membro não destrua a integridade das auditorias passadas.

Desenho Lógico da Base de Dados

4.1 Obtenção do Modelo Lógico

A transição do Modelo Conceptual (Entidade-Relacionamento) para o Modelo Lógico (Relacional) foi executada através do mapeamento sistemático de estruturas abstratas em tabelas lógicas, observando as regras formais de derivação de esquemas relacionais:

4.1.1 Mapeamento de Entidades Fortes

Cada entidade forte do modelo conceptual deu origem a uma tabela independente na qual os atributos simples foram convertidos em colunas. As chaves primárias conceptuais foram designadas como chaves primárias. (*Primary Keys - PK*).

4.1.2 Mapeamento de Relacionamentos 1:* (Um-para-Muitos)

Para propagar a semântica dos relacionamentos 1:* (como `Encarregado_Educacao` → `Jogador`), a chave primária da entidade do lado "1"foi exportada como chave estrangeira (*Foreign Key - FK*) para a tabela posicionada no lado "*". No caso específico do menor de idade, a coluna `id_enc_educacao` na tabela `Jogador` aceita valores nulos apenas se o atleta tiver idade igual ou superior a 18 anos, controlo este delegado para a fase de implementação física.

4.1.3 Mapeamento de Relacionamentos Muitos-para-Muitos (*:*)

De acordo com a metodologia de Connolly e Begg [0], os relacionamentos do tipo muitos-para-muitos (*:*) não podem ser implementados diretamente no modelo relacional. Para os representar, é obrigatória a criação de uma nova relação (tabela associativa) cuja chave primária é tipicamente composta pela junção das chaves estrangeiras das entidades originais participantes:

- **Tabela `Presenca`:** Resolve o relacionamento *:* entre `Jogador` e `Treino`, tendo uma chave primária composta por `(id_jogador, id_treino)`.
- **Tabela `Equipa_Tecnica`:** Resolve o relacionamento *:* entre `Treinador` e a entidade temporal `Escalao_Epoca`, com chave composta por `(id_treinador, id_escalao_epoca)`.

4.1.4 Mapeamento de Especialização (Herança de Alertas)

Para a entidade `Alerta` e as suas subentidades (`AlertaPagamento` e `AlertaPresenca`), o grupo optou pela estratégia de **Tabela por Subclasse**. Foi criada uma tabela para a entidade pai contendo os atributos genéricos (`id_alerta`, `mensagem`, `data_gerado`) e tabelas individuais para as subentidades. Nestas últimas, a chave primária desempenha simultaneamente o papel de chave estrangeira direcionada à tabela pai, garantindo uma extensão limpa e eliminando valores nulos generalizados.

4.1.5 Justificação das Decisões Estruturais do Grupo

- **Introdução do Atributo `ativo`:** Na transição para o modelo lógico, o grupo inseriu formalmente a coluna `ativo` (tipo lógico/booleano) nas tabelas `Jogador` e `Treinador`. Esta decisão técnica viabiliza o requisito de negócio de inativação de registos (eliminação lógica). Evita-se assim a instrução `DELETE` física, salvaguardando a integridade referencial do histórico de assiduidade e movimentos financeiros.

- **Relacionamento 1:1 Estrito (Quota - Pagamento):** Mapeou-se uma ligação ponderada de 1:1 entre `Quota` e `Pagamento`. Para o efetivar no modelo lógico, a tabela `Pagamento` recebeu a chave estrangeira `id_quota` com uma restrição de unicidade (*UNIQUE*). Isto impede matematicamente que uma quota receba mais do que um registo de liquidação, blindando o sistema contra dupla faturação.

4.2 Desenho do Modelo Lógico

Apresenta-se de seguida o desenho do Modelo Lógico final da base de dados, acompanhado da sua respetiva representação gráfica. Para uma visualização mais detalhada e em maior escala, o diagrama original encontra-se disponível nos Anexos. De salientar que as chaves primárias (PK) encontram-se sublinhadas e as chaves estrangeiras estão devidamente identificadas com a sigla FK (*Foreign Key*).

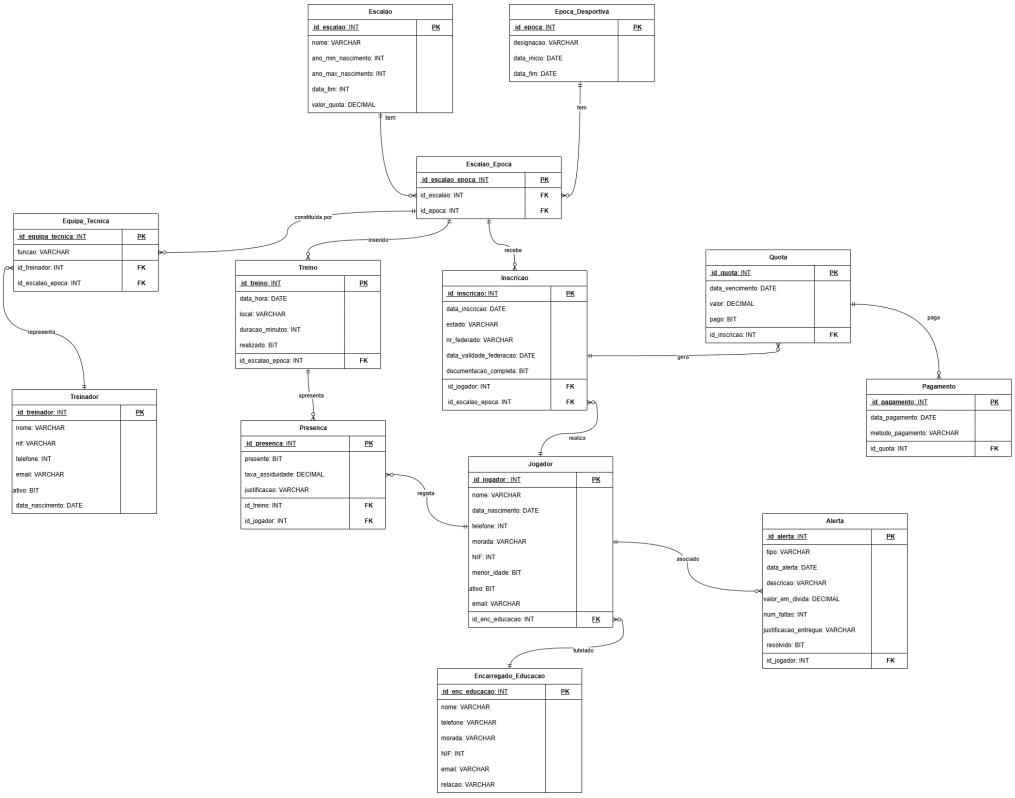


Figura 4.1: Diagrama do Modelo Lógico

4.2.1 Especificação do Desenho do Modelo Lógico

Para cada relação (tabela), estão identificados os respetivos atributos. As Chaves Primárias (PK) encontram-se sublinhadas e as dependências referenciais estão destacadas através da sigla *FK (Foreign Key)*, indicando a respetiva entidade de destino:

1. `Encarregado_Educacao` (id_enc_educacao, nome, telefone, email, morada, NIF, relacao)
2. `Jogador` (id_jogador, nome, telefone, data_nascimento, menor_idade, email, ativo, NIF, morada, id_enc_educacao)
   - *FK* `id_enc_educacao` → `Encarregado_Educacao`
3. `Escalao` (id_escalao, nome, ano_min_nascimento, ano_max_nascimento, data_fim, valor_quota)
4. `Epoca_Desportiva` (id_epoca, designacao, data_inicio, data_fim)
5. `Escalao_Epoca` (id_escalao_epoca, id_escalao, id_epoca)
   - *FK* `id_escalao` → `Escalao` (`id_escalao`)
   - *FK* `id_epoca` → `Epoca_Desportiva` (`id_epoca`)
6. `Inscricao` (id_inscricao, data_inscricao, estado, nr_federado,data_validade_federacao, documentacao_completa, id_jogador, id_escalao_epoca)
   - *FK* `id_jogador` → `Jogador`
   - *FK* `id_escalao_epoca` → `Escalao_Epoca`
7. `Quota` (id_quota, valor, data_vencimento, pago id_inscricao)
   - *FK* `id_inscricao` → `Inscricao`
8. `Pagamento` (id_pagamento, data_pagamento, metodo_pagamento, id_quota)
   - *FK* `id_quota` → `Quota`
9. `Treino` (id_treino, data_hora, local, duracao_minutos, realizado, id_escalao_epoca)
   - *FK* `id_escalao_epoca` → `Escalao_Epoca`
10. `Presenca` (id_presenca, presente, taxa_assiduidade, justificacao, id_treino, id_jogador)
    - *FK* `id_jogador` → `Jogador`
    - *FK* `id_treino` → `Treino`
11. `Treinador` (id_treinador, nome, nif, telefone, email, ativo, data_nascimento)
12. `Equipa_Tecnica` (id_equipa_tecnica, funcao, id_treinador, id_escalao_epoca)

- *FK* `id_treinador` → `Treinador`
- *FK* `id_escalao_epoca` → `Escalao_Epoca`

13. `Alerta` (id_alerta,tipo, descricao, data_alerta, valor_em_divida, num_faltas, justificacao_entregue, resolvido, id_jogador)
    - *FK* `id_jogador` → `Jogador`

4.3 Normalização

O processo de normalização foi aplicado para validar a qualidade do desenho do Modelo Lógico, garantindo a eliminação de anomalias de inserção, atualização e remoção de dados. O desenho foi submetido à verificação direta das três formas normais:

4.3.1 Primeira Forma Normal (1FN)

Uma tabela encontra-se na 1FN se todos os seus atributos forem atómicos (indivisíveis) e não existirem grupos repetitivos ou atributos com múltiplos valores.

- **Validação:** O modelo cumpre este critério ao eliminar grupos repetitivos. Como ilustrado na Figura 4.2, o documento original (Relatório de Treino) continha uma lista com os jogadores presentes (múltiplos valores para um só treino). Para resolver isto, essa lista foi "achatada", criando-se uma estrutura onde cada linha representa apenas a presença de um único jogador. Para garantir que não há repetições, a chave primária passou a ser composta por `id_treino` e `id_jogador`.

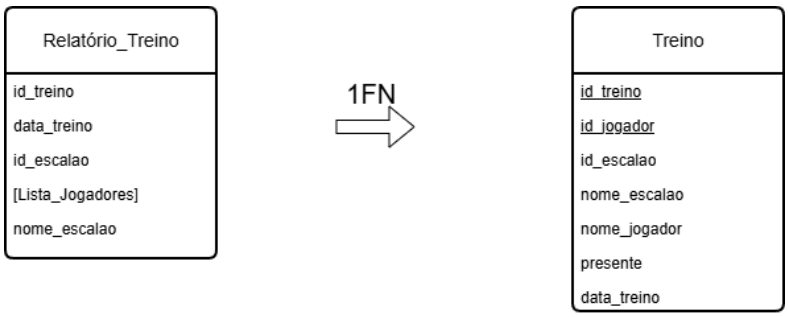


Figura 4.2: Exemplo de Aplicação da 1FN (Remoção de Grupos Repetitivos)

4.3.2 Segunda Forma Normal (2FN)

Uma tabela encontra-se na 2FN se já cumprir os requisitos da 1FN e se todos os seus atributos não-chave dependerem funcionalmente de forma total da chave primária, eliminando dependências parciais.

- **Validação:** O modelo cumpre a 2FN ao eliminar dependências parciais. Como ilustrado na Figura 4.3, partindo da tabela à esquerda (resultante do passo anterior), atributos como o `nome_jogador` dependiam apenas de uma parte da chave (`id_jogador`), enquanto dados como a `data_treino` dependiam apenas do `id_treino`. Para corrigir isto, dividimos a informação em três entidades distintas (Treino, Jogador e Presença), garantindo que em cada tabela os atributos dependem da totalidade da respetiva chave primária.

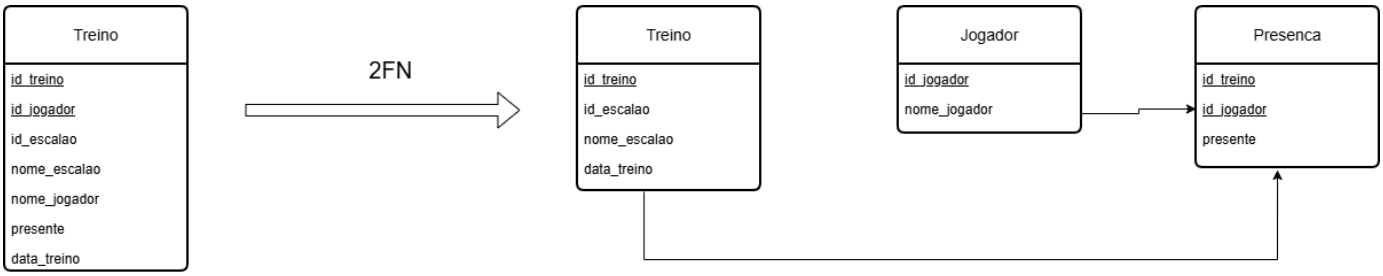


Figura 4.3: Exemplo de Aplicação da 2FN (Remoção de Dependências Parciais)

4.3.3 Terceira Forma Normal (3FN)

Uma tabela encontra-se na 3FN se estiver na 2FN e se nenhum dos seus atributos não-chave depender de forma transitiva da chave primária através de outro atributo não-chave. Ou seja, não podem existir dependências indiretas.

- **Validação:** O modelo atinge a 3FN ao remover dependências transitivas (indiretas). Observando a tabela de Treino criada na 2FN (Figura 4.4), verificamos que o `nome_escalao` dependia do `id_escalao`, e não da chave principal do treino. Para corrigir esta situação, o Escalão foi isolado numa nova entidade com a sua própria chave primária, deixando na tabela Treino apenas a chave estrangeira para fazer a ligação. Obtemos assim as quatro tabelas finais e perfeitamente normalizadas.

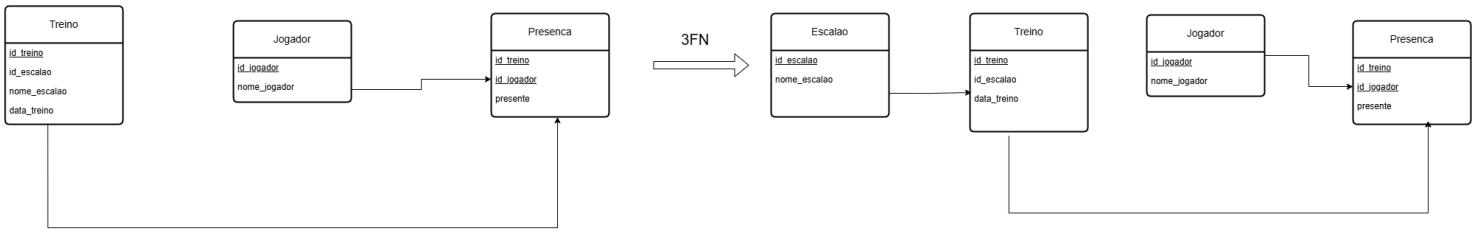


Figura 4.4: Exemplo de Aplicação da 3FN (Remoção de Dependências Transitivas)

Desenho Físico

Neste capítulo, apresenta-se a transição do modelo lógico para a sua implementação física num Sistema de Gestão de Bases de Dados Relacionais (SGBDR), concretizando as estruturas de armazenamento, regras de negócio e mecanismos de manipulação de dados através da linguagem SQL.

5.1 Criação de Tabelas e Restrições de Integridade

Nesta parte do projeto, o Modelo Relacional foi traduzido para a sua implementação física no Sistema de Gestão de Bases de Dados (SQL Server), utilizando a linguagem T-SQL. Durante a criação das tabelas, foi dada especial atenção à implementação de restrições de integridade (*constraints*) para garantir que a base de dados se "auto-protege"contra a inserção de dados inválidos ou inconsistentes.

5.1.1 Estrutura Base e Chaves Primárias

A definição da estrutura de cada tabela foi realizada através do comando `CREATE TABLE`. Para garantir a identificação única de cada registo, foram definidas Chaves Primárias (`PRIMARY KEY`) suportadas pela propriedade `IDENTITY(1,1)`. Esta abordagem permite ao motor de base de dados gerar identificadores numéricos de forma sequencial e automática (por exemplo, `id_jogador` ou `id_inscricao`), mitigando o risco de falhas humanas resultantes da numeração manual.

5.1.2 Integridade Referencial (Chaves Estrangeiras)

Para assegurar a coerência e a correta relação entre as várias entidades do sistema, impedindo a existência de "dados órfãos", foram implementadas restrições do tipo `FOREIGN KEY`. A título de exemplo, a tabela de Inscrições está diretamente vinculada ao Jogador e ao respetivo Escalão/Época. Isto impossibilita a associação de uma inscrição a um atleta inexistente ou a uma época que não se encontra configurada no sistema.

5.1.3 Restrições de Domínio e Regras de Negócio

As regras de negócio do clube desportivo foram traduzidas diretamente para o motor da base de dados através de diversas restrições ao nível das colunas:

- **Valores por Omissão (`DEFAULT`):** Automatização do preenchimento de campos padrão. Na tabela `Inscricao`, o atributo `estado` assume automaticamente o valor `'Pendente'`, e a `data_inscricao` regista o momento exato da inserção através da função `GETDATE()`.
- **Validação Lógica (`CHECK`):** Garantia de que os dados inseridos fazem sentido no contexto real. Por exemplo, a validação do formato do email do Encarregado de Educação ou a restrição de que o valor de uma quota introduzida não possui um valor negativo.
- **Obrigatoriedade e Unicidade (`NOT NULL` e `UNIQUE`):** Uma decisão arquitetural crucial foi tomada em relação ao atributo `nr_federado`. Este campo foi definido como estritamente obrigatório (`NOT NULL`) em conjunto com uma restrição de unicidade (`UNIQUE CONSTRAINT`). Esta implementação reflete uma regra administrativa rigorosa do clube: nenhum atleta pode ser registado sem a respetiva identificação federativa definitiva, garantindo a unicidade do número (evitando duplicações ou fraudes) e a total conformidade com a associação desportiva.

5.1.4 Exemplo de Implementação: Tabela de Inscrição

Na listagem abaixo, é possível observar a aplicação prática das várias restrições de integridade discutidas, ilustrando a robustez lógica implementada durante a criação física das tabelas do modelo.

Criação da tabela Inscricao com as respetivas restrições de integridade

```
CREATE TABLE Inscricao (
    id_inscricao          INT         IDENTITY(1,1) PRIMARY KEY,
    data_inscricao        DATE        NOT NULL DEFAULT GETDATE(),
    estado                VARCHAR(20) NOT NULL DEFAULT 'Pendente',
    nr_federado           VARCHAR(30) NOT NULL,
    data_validade_federacao DATE      NULL,
    documentacao_completa BIT         NOT NULL DEFAULT 0,
    id_jogador            INT         NOT NULL,
    id_escalao_epoca      INT         NOT NULL,
    CONSTRAINT FK_Inscricao_Jogador  FOREIGN KEY (id_jogador)
        REFERENCES Jogador(id_jogador),
    CONSTRAINT FK_Inscricao_EscalaoEpoca FOREIGN KEY (id_escalao_epoca)
        REFERENCES Escalao_Epoca(id_escalao_epoca),
    CONSTRAINT CK_Inscricao_Estado   CHECK (estado IN
        ('Pendente','Ativa','Suspensa','Cancelada')),
    CONSTRAINT UQ_Inscricao_JogadorEpoca UNIQUE (id_jogador,
         id_escalao_epoca),
    CONSTRAINT UQ_Inscricao_Federado UNIQUE (nr_federado)
);
GO
```

5.2 Views

Para este trabalho, foram implementadas diversas Vistas (*Views*) com o objetivo inicial de simplificar a extração de dados, abstrair a complexidade estrutural e fornecer "montras"de informação prontas a consumir pelos utilizadores finais (direção do clube, coordenadores e treinadores).

A utilização de *Views* neste projeto justificou-se pelos seguintes motivos técnicos e de negócio:

- **Abstração de Complexidade:** A maioria dos relatórios do clube exige o cruzamento de dados de várias tabelas (ex: `Jogador`, `Inscricao`, `Escalao_Epoca`, `Escalao`). As vistas encapsulam os múltiplos `JOIN`s, permitindo que o utilizador consulte a informação com um simples comando `SELECT * FROM NomeDaVista`.
- **Campos Calculados e Formatação:** Utilização de expressões lógicas (como o `CASE WHEN`) para transformar dados em bruto em informação percetível. Por exemplo, traduzir a ausência de uma data de validade na federação para o texto legível "Não Federado".
- **Segurança e Encapsulamento:** As vistas atuam como uma camada de segurança, permitindo restringir o acesso direto às tabelas base e expondo apenas as colunas e os registos estritamente necessários para cada contexto.
- **Agregações e Estatísticas:** Implementação de funções de agregação (como `COUNT` e `SUM`) para apresentar totais em tempo real, tais como o número de alertas por resolver de um jogador ou as quotas em atraso.

5.2.1 Exemplo de Implementação: Listagem de Jogadores por Escalão

Para ilustrar a aplicação destes conceitos, apresenta-se a vista `V_JogadoresInscritosEscalao`. Esta vista resolve uma das necessidades mais frequentes da secretaria do clube: consultar a lista de atletas de um determinado escalão e época, verificando instantaneamente a sua situação federativa.

Criação da vista para consulta de jogadores e respetivo estado federativo

```
CREATE OR ALTER VIEW V_JogadoresInscritosEscalao AS
SELECT j.nome AS Nome_Jogador, j.data_nascimento AS Data_Nascimento,
       e.nome AS Escalao, ep.designacao AS Epoca,
       insc.estado AS Estado_Inscricao, insc.nr_federado AS
           Numero_Federado,
       CASE
```

```
            WHEN insc.nr_federado IS NULL THEN 'Nao Federado'
            WHEN insc.data_validade_federacao IS NULL THEN 'Federado (sem
                validade)'
            WHEN insc.data_validade_federacao < GETDATE() THEN 'Federacao
                Expirada'
            ELSE 'Federacao Valida'
        END AS Estado_Federativo
FROM Jogador j
JOIN Inscricao insc    ON insc.id_jogador = j.id_jogador
JOIN Escalao_Epoca ee  ON ee.id_escalao_epoca = insc.id_escalao_epoca
JOIN Escalao e         ON e.id_escalao = ee.id_escalao
JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca
WHERE j.ativo = 1;
GO
```

Através desta implementação, para listar todos os atletas do escalão de Benjamins numa época específica, o utilizador apenas necessita de executar uma consulta simples (`SELECT * FROM V_JogadoresInscritosEscalao WHERE Escalao = 'Benjamins'`), transferindo todo o peso do processamento relacional e das validações de datas para o Motor de Base de Dados.

5.3 Stored Procedures

Para a manipulação de dados e execução de operações complexas, optou-se pela implementação de *Stored Procedures*. Esta abordagem afasta a lógica de negócio da camada aplicacional e centraliza-a diretamente no Motor de Base de Dados, garantindo maior robustez e consistência.

A adoção de *Stored Procedures* neste projeto justifica-se pelos seguintes fatores fundamentais:

- **Gestão de Transações:** Operações como o registo de um novo atleta ou o pagamento de uma quota envolvem a inserção e atualização simultânea de múltiplas tabelas. As *Stored Procedures* permitem encapsular estas ações em blocos transacionais (`BEGIN TRAN` e `COMMIT`). Se ocorrer um erro a meio do processo, o comando `ROLLBACK` reverte todas as alterações, impedindo

que a base de dados fique num estado inconsistente (por exemplo, registar o pagamento sem abater a dívida correspondente).

- **Centralização da Lógica de Negócio:** Processos demorados, como calcular a idade exata de um jogador no momento da inscrição e validar se o mesmo se enquadra nos limites de idades (`ano_min` e `ano_max`) do Escalão pretendido, são processados centralmente. O utilizador final apenas necessita de fornecer os dados básicos, deixando o processamento pesado para o servidor.
- **Segurança e Prevenção de Injeção SQL:** Ao obrigar a aplicação a interagir com a base de dados exclusivamente através da passagem de parâmetros para as *Procedures*, reduz-se drasticamente a superfície de ataque para vulnerabilidades como o *SQL Injection*.

5.3.1 Exemplo de Implementação: Registo de Inscrição de Jogador

Para ilustrar a aplicação destes conceitos na camada de base de dados, apresenta-se a *stored procedure* `sp_RegistarInscricaoJogador`. Este procedimento automatiza o processo de inscrição de um atleta num determinado escalão e época desportiva. O mecanismo garante o cumprimento rigoroso da regra de negócio relativa aos limites de idades (RN02), verificando se o ano de nascimento do jogador se enquadra nos anos mínimo e máximo definidos para o escalão pretendido. Caso a validação seja bem-sucedida, a inscrição é registada com o estado inicial de 'Pendente'; caso contrário, a operação é bloqueada e é devolvida uma mensagem de erro explicativa

sp_RegistarInscricaoJogador

```
CREATE OR ALTER PROCEDURE sp_RegistarInscricaoJogador
    @id_jogador          INT,
    @id_escalao_epoca    INT,
    @documentacao_completa BIT = 0
AS
BEGIN
    SET NOCOUNT ON;
```

```
    DECLARE @ano_nasc INT, @ano_min INT, @ano_max INT, @nome_esc
        VARCHAR(50);
    SELECT @ano_nasc = YEAR(data_nascimento) FROM Jogador WHERE
        id_jogador = @id_jogador;
    IF @ano_nasc IS NULL THROW 51001, 'Jogador nao encontrado.', 1;
    SELECT @ano_min = e.ano_min_nascimento, @ano_max =
        e.ano_max_nascimento,
          @nome_esc = e.nome
    FROM Escalao_Epoca ee JOIN Escalao e ON ee.id_escalao =
        e.id_escalao
    WHERE ee.id_escalao_epoca = @id_escalao_epoca;
    IF @ano_min IS NULL THROW 51002, 'Escalao/Epoca nao
        encontrado.', 1;
    IF @ano_nasc < @ano_min OR @ano_nasc > @ano_max
    BEGIN
        DECLARE @msg VARCHAR(250);
        SET @msg = CONCAT('RNO4: Ano nascimento (', @ano_nasc,
                          ') incompativel com ', @nome_esc,
                          ' (', @ano_min, '-', @ano_max, ').');
        THROW 51003, @msg, 1;
    END;
    INSERT INTO Inscricao (estado, data_inscricao,
        documentacao_completa,
                           id_jogador, id_escalao_epoca)
    VALUES ('Pendente', GETDATE(), @documentacao_completa,
        @id_jogador, @id_escalao_epoca);
END;
GO
```

5.4 Triggers

Os *Triggers* foram implementados para garantir regras de negócio complexas, reagindo automaticamente a qualquer manipulação de dados (`INSERT`, `UPDATE`, `DELETE`). Neste projeto, foram essenciais para assegurar os seguintes objetivos:

- **Validação entre Tabelas:** Permitem cruzar dados de tabelas diferentes. Exemplo: impedir que um escalão tenha mais de um "Treinador Principal"na mesma época. Como um simples `CHECK` não faz isto, o *Trigger* bloqueia a operação e devolve um erro personalizado.
- **Sistema Dinâmico:** O código não tem valores fixos. Na cobrança de quotas, o *Trigger* lê o `valor_quota` diretamente da tabela `Escalao`. Assim, a direção do clube pode alterar os preços na base de dados sem precisar de um programador para alterar o código.
- **Automatização de Regras:** Executam validações automáticas e cruciais. Exemplo: ao inserir um jogador, o sistema calcula a sua idade em tempo real e bloqueia a inscrição caso seja menor de idade e não tenha um Encarregado de Educação associado.
- **Segurança Absoluta:** Garantem que as regras do clube nunca são contornadas, quer os dados sejam inseridos através de um formulário de software ou diretamente no motor da base de dados por um administrador.

5.4.1 Exemplo de Implementação: Validação de Menores de Idade

Para ilustrar a aplicação destes conceitos, apresenta-se o *trigger* `TR_Jogador_Menoridade`. Este mecanismo garante o cumprimento automático de uma regra de negócio crítica para o clube (RN02): calcular e atualizar o estatuto de menoridade do atleta no momento do seu registo ou atualização, bloqueando a operação caso o jogador seja menor e não tenha um encarregado de educação associado. Sempre que existe uma atualização na tabela Jogador, o *Trigger* é acionado.

TR_Jogador_Menoridade

```
CREATE OR ALTER TRIGGER TR_Jogador_Menoridade
ON Jogador AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(data_nascimento) AND NOT UPDATE(id_enc_educacao)
        RETURN;
    UPDATE j SET menor_idade = CASE
        WHEN DATEDIFF(YEAR, i.data_nascimento, GETDATE()) < 18 THEN
            1 ELSE 0
    END FROM Jogador j JOIN inserted i ON j.id_jogador =
        i.id_jogador;
    IF EXISTS (SELECT 1 FROM inserted
              WHERE DATEDIFF(YEAR, data_nascimento, GETDATE()) < 18
              AND id_enc_educacao IS NULL)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50001, 'Validacao (RN15): Jogador menor requer
            Encarregado de Educacao.', 1;
    END;
END;
GO
```

Conclusões

O desenvolvimento deste projeto permitiu criar um sistema de base de dados robusto, seguro e totalmente funcional para a gestão de um clube desportivo. Os principais objetivos propostos foram alcançados com sucesso:

- **Modelação e Normalização:** O modelo de dados foi desenhado com rigor e validado até à 3ª Forma Normal (3FN), garantindo uma estrutura limpa e sem redundâncias.
- **Integridade e Regras de Negócio:** A implementação de restrições (*Constraints*) e *Triggers* garantiu que o sistema se auto-protege contra erros humanos e respeita as regras do clube (como o controlo de idades, estados federativos e quotas).
- **Eficiência no Acesso:** A criação de Vistas e *Stored Procedures* permitiu simplificar as consultas diárias e transferir a complexidade do processamento para o motor da base de dados.

No geral, o sistema desenvolvido reflete com precisão as necessidades reais da secretaria e equipa técnica do clube, centralizando a gestão de atletas, equipas, inscrições e treinos.

Bibliografia

Connolly, T., & Begg, C. (2014). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6ª ed.). Pearson Education.

Anexos

Anexo A — Diagrama do Modelo Lógico

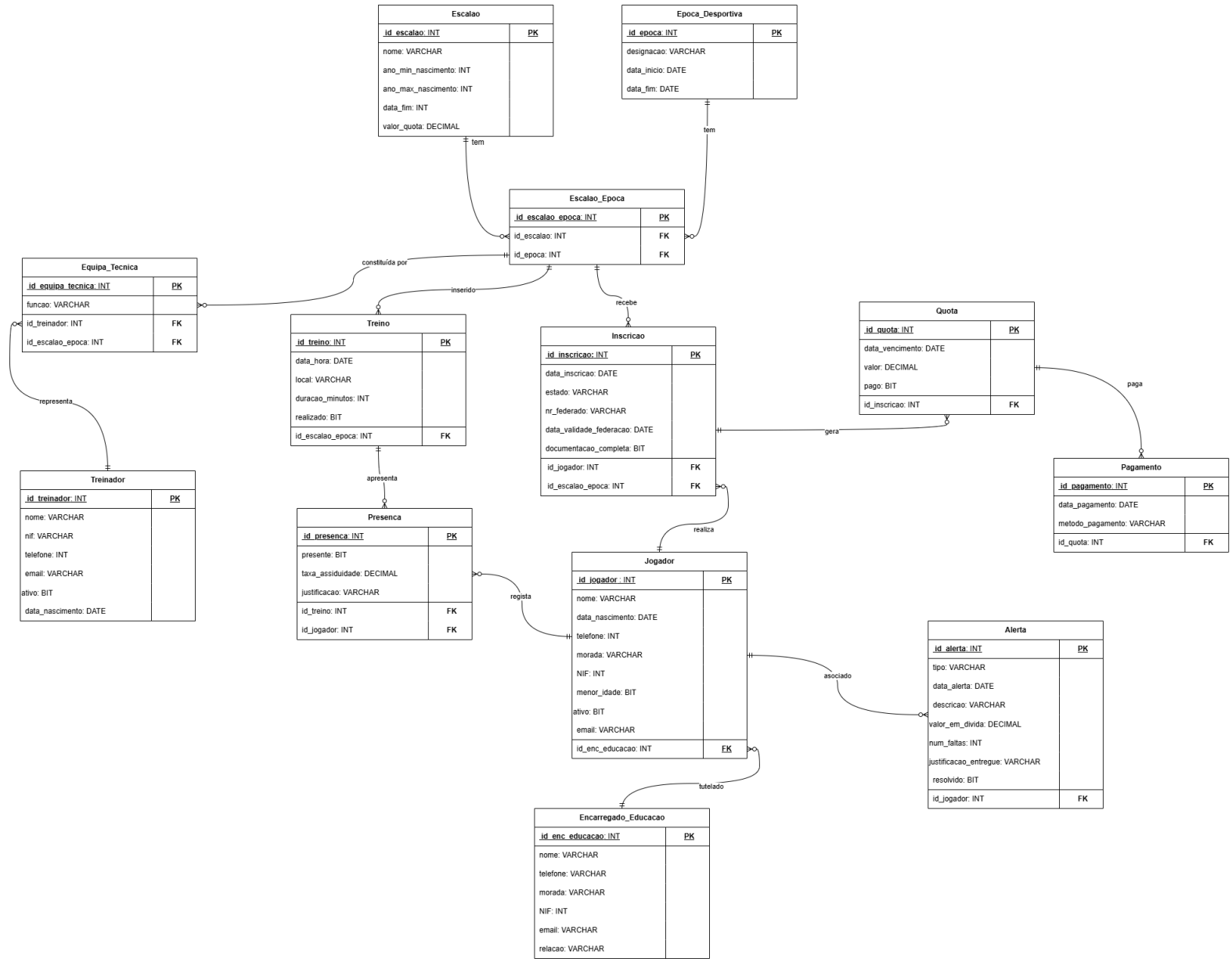


Figura 6.1: Diagrama do Modelo Lógico

Anexo B — Criação das Tabelas (DDL)

O script seguinte apresenta a criação completa das tabelas da base de dados, incluindo todas as restrições de integridade referencial e de domínio.

Criação das Tabelas – ClubeFormacaoDB

```
USE ClubeFormacaoDB;
GO

-- Encarregado_Educacao
CREATE TABLE Encarregado_Educacao (
    id_enc_educacao INT IDENTITY(1,1) PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    telefone        VARCHAR(9)  NOT NULL,
    email           VARCHAR(100) NULL,
    morada          VARCHAR(200) NULL,
    NIF             INT          NULL,
    relacao         VARCHAR(30) NOT NULL,
    CONSTRAINT CK_EE_Contacto CHECK (LEN(telefone) = 9 AND
        ISNUMERIC(telefone) = 1),
    CONSTRAINT CK_EE_Email  CHECK (email LIKE '%@%.%'),
    CONSTRAINT CK_EE_Relacao CHECK (relacao IN
        ('Pai','Mae','Avo/Avo','Tutor Legal','Outro')),
    CONSTRAINT UQ_EE_Telefone UNIQUE (telefone),
    CONSTRAINT UQ_EE_Email  UNIQUE (email)
);
GO

-- Jogador
CREATE TABLE Jogador (
    id_jogador     INT IDENTITY(1,1) PRIMARY KEY,
    nome           VARCHAR(100) NOT NULL,
    data_nascimento DATE       NOT NULL,
    telefone       INT         NULL,
    morada         VARCHAR(200) NULL,
    NIF            INT         NULL,
```

```
    menor_idade    BIT         NOT NULL,
    email          VARCHAR(100) NULL,
    id_enc_educacao INT        NULL,
    ativo          BIT         NOT NULL DEFAULT 1,
    CONSTRAINT FK_Jogador_EE FOREIGN KEY (id_enc_educacao)
        REFERENCES Encarregado_Educacao(id_enc_educacao),
    CONSTRAINT UQ_Jogador    UNIQUE (nome, data_nascimento),
    CONSTRAINT CK_Jogador_Idade CHECK (DATEDIFF(YEAR, data_nascimento,
        GETDATE()) >= 4),
    CONSTRAINT CK_Jogador_Nome CHECK (nome LIKE '% %' AND nome NOT LIKE
        '%[0-9]%')
);
GO

-- Escalao
CREATE TABLE Escalao (
    id_escalao       INT IDENTITY(1,1) PRIMARY KEY,
    nome             VARCHAR(50) NOT NULL,
    ano_min_nascimento INT       NOT NULL,
    ano_max_nascimento INT       NOT NULL,
    data_fim         INT         NULL,
    valor_quota      DECIMAL(5,2) NOT NULL
);
GO

-- Epoca_Desportiva
CREATE TABLE Epoca_Desportiva (
    id_epoca   INT IDENTITY(1,1) PRIMARY KEY,
    designacao VARCHAR(20) NOT NULL,
    data_inicio DATE      NOT NULL,
    data_fim   DATE       NOT NULL
);
GO

-- Escalao_Epoca
CREATE TABLE Escalao_Epoca (
```

```
    id_escalao_epoca INT IDENTITY(1,1) PRIMARY KEY,
    id_escalao      INT NOT NULL,
    id_epoca        INT NOT NULL,
    CONSTRAINT FK_EE_Escalao FOREIGN KEY (id_escalao)
        REFERENCES Escalao(id_escalao),
    CONSTRAINT FK_EE_Epoca FOREIGN KEY (id_epoca)
        REFERENCES Epoca_Desportiva(id_epoca)
);
GO

-- Inscricao
CREATE TABLE Inscricao (
    id_inscricao          INT         IDENTITY(1,1) PRIMARY KEY,
    data_inscricao        DATE        NOT NULL DEFAULT GETDATE(),
    estado                VARCHAR(20) NOT NULL DEFAULT 'Pendente',
    nr_federado           VARCHAR(30) NOT NULL,
    data_validade_federacao DATE      NULL,
    documentacao_completa BIT         NOT NULL DEFAULT 0,
    id_jogador            INT         NOT NULL,
    id_escalao_epoca      INT         NOT NULL,
    CONSTRAINT FK_Inscricao_Jogador   FOREIGN KEY (id_jogador)
        REFERENCES Jogador(id_jogador),
    CONSTRAINT FK_Inscricao_EscalaoEpoca FOREIGN KEY (id_escalao_epoca)
        REFERENCES Escalao_Epoca(id_escalao_epoca),
    CONSTRAINT CK_Inscricao_Estado    CHECK (estado IN
        ('Pendente','Ativa','Suspensa','Cancelada')),
    CONSTRAINT UQ_Inscricao_JogadorEpoca UNIQUE (id_jogador,
         id_escalao_epoca),
    CONSTRAINT UQ_Inscricao_Federado UNIQUE (nr_federado)
);
GO

-- Quota
CREATE TABLE Quota (
    id_quota       INT          IDENTITY(1,1) PRIMARY KEY,
    data_vencimento DATE        NOT NULL,
```

```
    valor          DECIMAL(5,2) NOT NULL,
    pago           BIT          NOT NULL DEFAULT 0,
    id_inscricao   INT          NOT NULL,
    CONSTRAINT FK_Quota_Inscricao FOREIGN KEY (id_inscricao)
        REFERENCES Inscricao(id_inscricao),
    CONSTRAINT CK_Quota_Valor    CHECK (valor > 30 AND valor <= 150),
    CONSTRAINT UQ_Quota_Mensalidade UNIQUE (id_inscricao,
         data_vencimento)
);
GO

-- Pagamento
CREATE TABLE Pagamento (
    id_pagamento    INT          IDENTITY(1,1) PRIMARY KEY,
    data_pagamento  DATE         NOT NULL,
    valor_pago      DECIMAL(5,2) NOT NULL,
    metodo_pagamento VARCHAR(30) NOT NULL,
    id_quota        INT          NOT NULL,
    CONSTRAINT FK_Pagamento_Quota FOREIGN KEY (id_quota)
        REFERENCES Quota(id_quota),
    CONSTRAINT CK_Pagamento_Metodo CHECK (metodo_pagamento IN
        ('MB Way','Multibanco','Transferencia','Numerario')),
    CONSTRAINT CK_Pagamento_Data CHECK (data_pagamento <= CAST(GETDATE()
         AS DATE)),
    CONSTRAINT UQ_Pagamento_Quota UNIQUE (id_quota)
);
GO

-- Treinador
CREATE TABLE Treinador (
    id_treinador   INT          IDENTITY(1,1) PRIMARY KEY,
    nome           VARCHAR(100) NOT NULL,
    nif            VARCHAR(9)   NULL,
    telefone       INT          NULL,
    email          VARCHAR(100) NULL,
    ativo          BIT          NOT NULL DEFAULT 1,
```

```
    data_nascimento DATE      NULL
);
GO

-- Treino
CREATE TABLE Treino (
    id_treino       INT         IDENTITY(1,1) PRIMARY KEY,
    data_hora       DATETIME    NOT NULL,
    local           VARCHAR(100) NOT NULL,
    duracao_minutos INT         NOT NULL,
    realizado       BIT         NOT NULL DEFAULT 0,
    id_escalao_epoca INT        NOT NULL,
    CONSTRAINT FK_Treino_EscalaoEpoca FOREIGN KEY (id_escalao_epoca)
        REFERENCES Escalao_Epoca(id_escalao_epoca),
    CONSTRAINT CK_Treino_Horario   CHECK (CAST(data_hora AS TIME)
        BETWEEN '08:00:00' AND '22:30:00'),
    CONSTRAINT CK_Treino_Duracao   CHECK (duracao_minutos >= 30)
);
GO

-- Presenca
CREATE TABLE Presenca (
    id_presenca     INT         IDENTITY(1,1) PRIMARY KEY,
    presente        BIT         NOT NULL DEFAULT 0,
    taxa_assiduidade DECIMAL(5,2) NOT NULL DEFAULT 0,
    convocado       BIT         NOT NULL DEFAULT 0,
    justificacao    VARCHAR(200) NULL,
    id_treino       INT         NOT NULL,
    id_jogador      INT         NOT NULL,
    CONSTRAINT FK_Presenca_Treino FOREIGN KEY (id_treino)
        REFERENCES Treino(id_treino),
    CONSTRAINT FK_Presenca_Jogador FOREIGN KEY (id_jogador)
        REFERENCES Jogador(id_jogador),
    CONSTRAINT CK_Presenca_Taxa CHECK (taxa_assiduidade BETWEEN 0 AND
        100),
    CONSTRAINT UQ_Presenca      UNIQUE (id_jogador, id_treino)
```

```
);
GO

-- Equipa_Tecnica
CREATE TABLE Equipa_Tecnica (
    id_equipa_tecnica INT      IDENTITY(1,1) PRIMARY KEY,
    funcao            VARCHAR(30) NOT NULL,
    id_treinador      INT        NOT NULL,
    id_escalao_epoca  INT        NOT NULL,
    CONSTRAINT FK_ET_Treinador   FOREIGN KEY (id_treinador)
        REFERENCES Treinador(id_treinador),
    CONSTRAINT FK_ET_EscalaoEpoca FOREIGN KEY (id_escalao_epoca)
        REFERENCES Escalao_Epoca(id_escalao_epoca),
    CONSTRAINT CK_ET_Funcao      CHECK (funcao IN
        ('Treinador Principal','Adjunto','Treinador GR','Preparador
            Fisico')),
    CONSTRAINT UQ_ET_TreinadorEpoca UNIQUE (id_treinador,
         id_escalao_epoca)
);
GO

-- Alerta
CREATE TABLE Alerta (
    id_alerta           INT         IDENTITY(1,1) PRIMARY KEY,
    tipo                VARCHAR(50) NOT NULL,
    data_alerta         DATE        NOT NULL DEFAULT GETDATE(),
    descricao           VARCHAR(300) NULL,
    valor_em_divida     DECIMAL(5,2) NULL,
    num_faltas          INT         NULL,
    justificacao_entregue VARCHAR(200) NULL,
    resolvido           BIT         NOT NULL DEFAULT 0,
    id_jogador          INT         NOT NULL,
    CONSTRAINT FK_Alerta_Jogador FOREIGN KEY (id_jogador)
        REFERENCES Jogador(id_jogador)
);
GO
```

Anexo C — Triggers

Os *triggers* implementados garantem as regras de negócio complexas que não podem ser asseguradas por simples *constraints*.

C.1 Tabela: Jogador

TR_Jogador_Menoridade

```
CREATE OR ALTER TRIGGER TR_Jogador_Menoridade
ON Jogador AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(data_nascimento) AND NOT UPDATE(id_enc_educacao)
        RETURN;
    UPDATE j SET menor_idade = CASE
        WHEN DATEDIFF(YEAR, i.data_nascimento, GETDATE()) < 18 THEN 1
            ELSE 0
    END FROM Jogador j JOIN inserted i ON j.id_jogador = i.id_jogador;
    IF EXISTS (SELECT 1 FROM inserted
              WHERE DATEDIFF(YEAR, data_nascimento, GETDATE()) < 18
              AND id_enc_educacao IS NULL)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50001, 'Validacao (RN15): Jogador menor requer Encarregado
            de Educacao.', 1;
    END;
END;
GO
```

TR_Jogador_SoftDelete

```
CREATE OR ALTER TRIGGER TR_Jogador_SoftDelete
ON Jogador INSTEAD OF DELETE
AS
```

```
BEGIN
    SET NOCOUNT ON;
    UPDATE Jogador SET ativo = 0
    WHERE id_jogador IN (SELECT id_jogador FROM deleted);
END;
GO
```

TR_Jogador_ValidarIdade

```
CREATE OR ALTER TRIGGER TR_Jogador_ValidarIdade
ON Jogador AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted
              WHERE DATEDIFF(YEAR, data_nascimento, GETDATE()) > 19)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50021, 'Validacao (RN03): Clube apenas aceita jogadores
            ate 19 anos.', 1;
    END;
END;
GO
```

C.2 Tabela: Encarregado_Educacao

TR_EncarregadoEducacao_BloqueioRemocao

```
CREATE OR ALTER TRIGGER TR_EncarregadoEducacao_BloqueioRemocao
ON Encarregado_Educacao INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM deleted d
```

```
        JOIN Jogador j     ON j.id_enc_educacao = d.id_enc_educacao AND
            j.ativo = 1
        JOIN Inscricao insc ON insc.id_jogador = j.id_jogador AND
            insc.estado = 'Ativa'
        WHERE DATEDIFF(YEAR, j.data_nascimento, GETDATE()) < 18)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50002, 'Validacao (RN19): Encarregado tem menores com
            inscricao ativa.', 1;
    END;
    ELSE
        DELETE FROM Encarregado_Educacao
        WHERE id_enc_educacao IN (SELECT id_enc_educacao FROM deleted);
END;
GO
```

C.3 Tabela: Inscricao

TR_Inscricao_EstadoPendente

```
CREATE OR ALTER TRIGGER TR_Inscricao_EstadoPendente
ON Inscricao AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Inscricao SET estado = 'Pendente'
    WHERE id_inscricao IN (SELECT id_inscricao FROM inserted);
END;
GO
```

TR_Inscricao_Ativacao

```
CREATE OR ALTER TRIGGER TR_Inscricao_Ativacao
ON Inscricao AFTER UPDATE
AS
BEGIN
```

```
    SET NOCOUNT ON;
    IF NOT UPDATE(estado) RETURN;
    INSERT INTO Quota (data_vencimento, valor, pago, id_inscricao)
    SELECT EOMONTH(GETDATE()), 35.00, 0, i.id_inscricao
    FROM inserted i JOIN deleted d ON i.id_inscricao = d.id_inscricao
    WHERE i.estado = 'Ativa' AND d.estado <> 'Ativa'
      AND NOT EXISTS (SELECT 1 FROM Quota q
                      WHERE q.id_inscricao = i.id_inscricao
                      AND q.data_vencimento = EOMONTH(GETDATE()));
END;
GO
```

TR_Inscricao_ValidarRegras

```
CREATE OR ALTER TRIGGER TR_Inscricao_ValidarRegras
ON Inscricao AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    -- RN04: Compatibilidade etaria
    IF EXISTS (
        SELECT 1 FROM inserted i
        JOIN Jogador j    ON j.id_jogador = i.id_jogador AND j.ativo = 1
        JOIN Escalao_Epoca ee ON ee.id_escalao_epoca = i.id_escalao_epoca
        JOIN Escalao e    ON e.id_escalao = ee.id_escalao
        WHERE YEAR(j.data_nascimento) < e.ano_min_nascimento
           OR YEAR(j.data_nascimento) > e.ano_max_nascimento)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50004, 'Validacao (RN04): Idade incompativel com
            escalao.', 1;
    END;
    -- RN05: Quotas em divida
    IF UPDATE(estado)
    BEGIN
        IF EXISTS (
```

```
            SELECT 1 FROM inserted i JOIN deleted d ON i.id_inscricao =
                d.id_inscricao
            JOIN Inscricao insc_ant ON insc_ant.id_jogador = i.id_jogador
                                    AND insc_ant.id_inscricao <>
                                        i.id_inscricao
            JOIN Quota q ON q.id_inscricao = insc_ant.id_inscricao
                        AND q.pago = 0 AND q.data_vencimento < GETDATE()
            WHERE i.estado = 'Ativa' AND d.estado <> 'Ativa')
        BEGIN
            ROLLBACK TRANSACTION;
            THROW 50005, 'Validacao (RN05): Jogador tem quotas em
                divida.', 1;
        END;
    END;
END;
GO
```

TR_Inscricao_LimiteJogadores

```
CREATE OR ALTER TRIGGER TR_Inscricao_LimiteJogadores
ON Inscricao AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(estado) RETURN;
    IF EXISTS (
        SELECT 1 FROM inserted i JOIN deleted d ON i.id_inscricao =
            d.id_inscricao
        WHERE i.estado = 'Ativa' AND d.estado <> 'Ativa'
          AND (SELECT COUNT(*) FROM Inscricao insc
               JOIN Jogador j ON j.id_jogador = insc.id_jogador AND
                   j.ativo = 1
               WHERE insc.id_escalao_epoca = i.id_escalao_epoca
               AND insc.estado = 'Ativa') > 25)
    BEGIN
        ROLLBACK TRANSACTION;
```

```
        THROW 50006, 'Validacao (RN06): Escalao atingiu limite de 25
            jogadores.', 1;
    END;
END;
GO
```

TR_Inscricao_AlertaFederacao

```
CREATE OR ALTER TRIGGER TR_Inscricao_AlertaFederacao
ON Inscricao AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(data_validade_federacao) RETURN;
    INSERT INTO Alerta (tipo, data_alerta, descricao, resolvido,
        id_jogador)
    SELECT 'Federacao Expirada', GETDATE(),
          'A inscricao federativa expirou e necessita de renovacao.', 0,
              i.id_jogador
    FROM inserted i JOIN Jogador j ON j.id_jogador = i.id_jogador AND
        j.ativo = 1
    WHERE i.data_validade_federacao < GETDATE()
      AND NOT EXISTS (SELECT 1 FROM Alerta a
                      WHERE a.id_jogador = i.id_jogador
                      AND a.tipo = 'Federacao Expirada' AND a.resolvido =
                          0);
END;
GO
```

TR_Inscricao_ValidarFederacao

```
CREATE OR ALTER TRIGGER TR_Inscricao_ValidarFederacao
ON Inscricao AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted
```

```
            WHERE nr_federado IS NOT NULL AND data_validade_federacao
                IS NULL)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50016, 'Validacao: Data validade federacao obrigatoria
            quando
          nr_federado esta preenchido.', 1;
    END;
END;
GO
```

C.4 Tabela: Quota

TR_Quota_BloqueioInscricaoCancelada

```
CREATE OR ALTER TRIGGER TR_Quota_BloqueioInscricaoCancelada
ON Quota AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted i
               JOIN Inscricao insc ON insc.id_inscricao = i.id_inscricao
               WHERE insc.estado = 'Cancelada')
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50007, 'Validacao: Quota nao pode ser gerada para
            inscricao Cancelada.', 1;
    END;
END;
GO
```

C.5 Tabela: Pagamento

TR_Pagamento_ValorExato

```
CREATE OR ALTER TRIGGER TR_Pagamento_ValorExato
ON Pagamento AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted i JOIN Quota q ON q.id_quota =
        i.id_quota
            WHERE i.valor_pago <> q.valor)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50008, 'Validacao (RN10): Valor pago nao corresponde ao
            valor da quota.', 1;
    END;
    UPDATE Quota SET pago = 1 WHERE id_quota IN (SELECT id_quota FROM
        inserted);
END;
GO
```

C.6 Tabela: Treino

TR_Treino_CriarPresencas

```
CREATE OR ALTER TRIGGER TR_Treino_CriarPresencas
ON Treino AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Presenca (presente, taxa_assiduidade, convocado,
        id_treino, id_jogador)
    SELECT 0, 0.00, 0, t.id_treino, i.id_jogador
    FROM inserted t JOIN Inscricao i ON i.id_escalao_epoca =
        t.id_escalao_epoca
                                AND i.estado = 'Ativa'
    WHERE NOT EXISTS (SELECT 1 FROM Presenca p
                    WHERE p.id_treino = t.id_treino AND p.id_jogador =
                        i.id_jogador);
```

```
END;
GO
```

TR_Treino_BloqueioRealizacaoFutura

```
CREATE OR ALTER TRIGGER TR_Treino_BloqueioRealizacaoFutura
ON Treino AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted i JOIN deleted d ON i.id_treino =
        d.id_treino
            WHERE i.realizado = 1 AND d.realizado = 0 AND i.data_hora
                > GETDATE())
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50009, 'Validacao: Nao e possivel marcar treino futuro
            como realizado.', 1;
    END;
END;
GO
```

TR_Treino_ChoqueCampo

```
CREATE OR ALTER TRIGGER TR_Treino_ChoqueCampo
ON Treino AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted i JOIN Treino t
            ON t.local = i.local AND t.data_hora = i.data_hora
            AND t.id_treino <> i.id_treino AND t.id_escalao_epoca <>
                i.id_escalao_epoca)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50017, 'Validacao (RN19): Campo ja ocupado neste
            horario.', 1;
```

```
    END;
END;
GO
```

C.7 Tabela: Presenca

TR_Presenca_ValidarTreino

```
CREATE OR ALTER TRIGGER TR_Presenca_ValidarTreino
ON Presenca AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(presente) RETURN;
    IF EXISTS (SELECT 1 FROM inserted i JOIN Treino t ON t.id_treino =
        i.id_treino
              WHERE i.presente = 1 AND (t.realizado = 0 OR t.data_hora >
                  GETDATE()))
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Validacao: Nao e permitido registar presenca em
            treino nao realizado.', 1;
    END;
END;
GO
```

TR_Presenca_SeguroAtivo

```
CREATE OR ALTER TRIGGER TR_Presenca_SeguroAtivo
ON Presenca AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(presente) RETURN;
    IF EXISTS (
        SELECT 1 FROM inserted i
```

```
        JOIN Jogador j    ON j.id_jogador = i.id_jogador AND j.ativo = 1
        JOIN Treino t     ON t.id_treino = i.id_treino
        JOIN Inscricao insc ON insc.id_jogador = i.id_jogador
                        AND insc.id_escalao_epoca = t.id_escalao_epoca
        WHERE i.presente = 1 AND insc.estado <> 'Ativa' AND i.convocado
            = 0)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50011, 'Validacao (RN14): Jogador sem inscricao ativa nao
            pode ter presenca.', 1;
    END;
END;
GO
```

TR_Presenca_CalcularAssiduidade

```
CREATE OR ALTER TRIGGER TR_Presenca_CalcularAssiduidade
ON Presenca AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(presente) RETURN;
    UPDATE p_alvo SET taxa_assiduidade = stats.nova_taxa
    FROM Presenca p_alvo
    JOIN (SELECT p.id_jogador,
                CAST(SUM(CAST(p.presente AS INT)) * 100.0
                / NULLIF(COUNT(*), 0) AS DECIMAL(5,2)) AS nova_taxa
          FROM Presenca p JOIN Jogador j ON j.id_jogador = p.id_jogador
              AND j.ativo = 1
          JOIN Treino t ON t.id_treino = p.id_treino
          JOIN Escalao_Epoca ee ON ee.id_escalao_epoca =
              t.id_escalao_epoca
          JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca AND
              ep.data_fim >= GETDATE()
          WHERE p.convocado = 0 AND p.id_jogador IN (SELECT id_jogador
              FROM inserted)
```

```
        GROUP BY p.id_jogador) AS stats ON stats.id_jogador =
            p_alvo.id_jogador
    WHERE p_alvo.id_jogador IN (SELECT id_jogador FROM inserted) AND
        p_alvo.convocado = 0;

    INSERT INTO Alerta (tipo, data_alerta, descricao, resolvido,
        id_jogador)
    SELECT DISTINCT 'Assiduidade Baixa', GETDATE(),
          'Taxa de assiduidade desceu abaixo de 50%.', 0, i.id_jogador
    FROM inserted i JOIN Presenca p ON p.id_jogador = i.id_jogador
                                AND p.id_presenca = i.id_presenca
    WHERE p.taxa_assiduidade < 50 AND p.convocado = 0
      AND NOT EXISTS (SELECT 1 FROM Alerta a WHERE a.id_jogador =
          i.id_jogador
                    AND a.tipo = 'Assiduidade Baixa' AND a.resolvido =
                        0);
END;
GO
```

C.8 Tabela: Equipa_Tecnica

TR_Treinador_SoftDelete

```
CREATE OR ALTER TRIGGER TR_Treinador_SoftDelete
ON Treinador INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Treinador SET ativo = 0
    WHERE id_treinador IN (SELECT id_treinador FROM deleted);
END;
GO
```

TR_EquipaTecnica_TreinadorPrincipal

```
CREATE OR ALTER TRIGGER TR_EquipaTecnica_TreinadorPrincipal
```

```
ON Equipa_Tecnica AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM inserted i
        JOIN Treinador tr    ON tr.id_treinador = i.id_treinador AND
            tr.ativo = 1
        JOIN Equipa_Tecnica et ON et.id_escalao_epoca =
            i.id_escalao_epoca
                            AND et.funcao = 'Treinador Principal'
                            AND et.id_equipa_tecnica <>
                                i.id_equipa_tecnica
        WHERE i.funcao = 'Treinador Principal')
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50014, 'Validacao (RN17): Escalao ja tem Treinador
            Principal.', 1;
    END;
END;
GO
```

TR_EquipaTecnica_TreinadorUnico

```
CREATE OR ALTER TRIGGER TR_EquipaTecnica_TreinadorUnico
ON Equipa_Tecnica AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM inserted i
        JOIN Treinador tr    ON tr.id_treinador = i.id_treinador AND
            tr.ativo = 1
        JOIN Equipa_Tecnica et ON et.id_treinador = i.id_treinador
                            AND et.id_equipa_tecnica <>
                                i.id_equipa_tecnica
```

```
        JOIN Escalao_Epoca ee1 ON ee1.id_escalao_epoca =
            i.id_escalao_epoca
        JOIN Escalao_Epoca ee2 ON ee2.id_escalao_epoca =
            et.id_escalao_epoca
                        AND ee2.id_epoca = ee1.id_epoca)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50015, 'Validacao (RN18): Treinador ja pertence a outra
            equipa nesta epoca.', 1;
    END;
END;
GO
```

Anexo D — Views

As *views* foram criadas para simplificar o acesso à informação e abstrair a complexidade das junções entre tabelas.

V_JogadoresInscritosEscalao

```
CREATE OR ALTER VIEW V_JogadoresInscritosEscalao AS
SELECT j.nome AS Nome_Jogador, j.data_nascimento AS Data_Nascimento,
       e.nome AS Escalao, ep.designacao AS Epoca,
       insc.estado AS Estado_Inscricao, insc.nr_federado AS
           Numero_Federado,
       CASE
           WHEN insc.nr_federado IS NULL THEN 'Nao Federado'
           WHEN insc.data_validade_federacao IS NULL THEN 'Federado (sem
               validade)'
           WHEN insc.data_validade_federacao < GETDATE() THEN 'Federacao
               Expirada'
           ELSE 'Federacao Valida'
       END AS Estado_Federativo
FROM Jogador j
JOIN Inscricao insc    ON insc.id_jogador = j.id_jogador
JOIN Escalao_Epoca ee  ON ee.id_escalao_epoca = insc.id_escalao_epoca
JOIN Escalao e         ON e.id_escalao = ee.id_escalao
JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca
WHERE j.ativo = 1;
GO
```

V_QuotasEmAtraso

```
CREATE OR ALTER VIEW V_QuotasEmAtraso AS
SELECT j.nome AS Nome_Jogador, e.nome AS Escalao, ep.designacao AS Epoca,
       q.data_vencimento AS Data_Vencimento, q.valor AS Valor_Em_Divida,
       DATEDIFF(DAY, q.data_vencimento, GETDATE()) AS Dias_Em_Atraso
FROM Jogador j
JOIN Inscricao insc    ON insc.id_jogador = j.id_jogador
JOIN Escalao_Epoca ee  ON ee.id_escalao_epoca = insc.id_escalao_epoca
```

```
JOIN Escalao e         ON e.id_escalao = ee.id_escalao
JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca
JOIN Quota q           ON q.id_inscricao = insc.id_inscricao
LEFT JOIN Pagamento p  ON p.id_quota = q.id_quota
WHERE j.ativo = 1 AND q.data_vencimento < CAST(GETDATE() AS DATE)
  AND p.id_pagamento IS NULL;
GO
```

V_AlertasAbertos

```
CREATE OR ALTER VIEW V_AlertasAbertos AS
SELECT a.tipo AS Tipo_Alerta, a.data_alerta AS Data_Alerta,
       a.descricao AS Descricao, j.nome AS Nome_Jogador,
       e.nome AS Escalao, ep.designacao AS Epoca,
       DATEDIFF(DAY, a.data_alerta, GETDATE()) AS Dias_Em_Aberto
FROM Alerta a
JOIN Jogador j             ON j.id_jogador = a.id_jogador
LEFT JOIN Inscricao insc   ON insc.id_jogador = j.id_jogador AND
    insc.estado = 'Ativa'
LEFT JOIN Escalao_Epoca ee ON ee.id_escalao_epoca = insc.id_escalao_epoca
LEFT JOIN Escalao e        ON e.id_escalao = ee.id_escalao
LEFT JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca
WHERE a.resolvido = 0;
GO
```

V_EquipaTecnicaCompleta

```
CREATE OR ALTER VIEW V_EquipaTecnicaCompleta AS
SELECT e.nome AS Escalao, ep.designacao AS Epoca,
       tr.nome AS Nome_Treinador, et.funcao AS Funcao,
       COUNT(et.id_equipa_tecnica) OVER (PARTITION BY
           et.id_escalao_epoca)
          AS Total_Membros_Equipa
FROM Equipa_Tecnica et
JOIN Treinador tr      ON tr.id_treinador = et.id_treinador AND tr.ativo
    = 1
JOIN Escalao_Epoca ee  ON ee.id_escalao_epoca = et.id_escalao_epoca
```

```
JOIN Escalao e         ON e.id_escalao = ee.id_escalao
JOIN Epoca_Desportiva ep ON ep.id_epoca = ee.id_epoca;
GO
```

Anexo E — Stored Procedures

As *stored procedures* encapsulam operações complexas que envolvem múltiplas tabelas, garantindo a consistência transacional.

sp_RegistarInscricaoJogador

```
CREATE OR ALTER PROCEDURE sp_RegistarInscricaoJogador
    @id_jogador          INT,
    @id_escalao_epoca    INT,
    @documentacao_completa BIT = 0
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ano_nasc INT, @ano_min INT, @ano_max INT, @nome_esc
        VARCHAR(50);
    SELECT @ano_nasc = YEAR(data_nascimento) FROM Jogador WHERE
        id_jogador = @id_jogador;
    IF @ano_nasc IS NULL THROW 51001, 'Jogador nao encontrado.', 1;
    SELECT @ano_min = e.ano_min_nascimento, @ano_max =
        e.ano_max_nascimento,
          @nome_esc = e.nome
    FROM Escalao_Epoca ee JOIN Escalao e ON ee.id_escalao = e.id_escalao
    WHERE ee.id_escalao_epoca = @id_escalao_epoca;
    IF @ano_min IS NULL THROW 51002, 'Escalao/Epoca nao encontrado.', 1;
    IF @ano_nasc < @ano_min OR @ano_nasc > @ano_max
    BEGIN
        DECLARE @msg VARCHAR(250);
        SET @msg = CONCAT('RN04: Ano nascimento (', @ano_nasc,
                          ') incompativel com ', @nome_esc,
                          ' (', @ano_min, '-', @ano_max, ').');
        THROW 51003, @msg, 1;
    END;
    INSERT INTO Inscricao (estado, data_inscricao, documentacao_completa,
                           id_jogador, id_escalao_epoca)
    VALUES ('Pendente', GETDATE(), @documentacao_completa, @id_jogador,
        @id_escalao_epoca);
```

```
END;
GO
```

sp_RegistarPresencaTreino

```
CREATE OR ALTER PROCEDURE sp_RegistarPresencaTreino
    @id_treino INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @id_escalao_epoca INT;
    SELECT @id_escalao_epoca = id_escalao_epoca FROM Treino WHERE
        id_treino = @id_treino;
    IF @id_escalao_epoca IS NULL THROW 52001, 'Treino nao encontrado.',
        1;
    UPDATE Treino SET realizado = 1 WHERE id_treino = @id_treino;
    INSERT INTO Presenca (presente, taxa_assiduidade, convocado,
        id_treino, id_jogador)
    SELECT 0, 0.00, 1, @id_treino, i.id_jogador
    FROM Inscricao i
    WHERE i.id_escalao_epoca = @id_escalao_epoca AND i.estado = 'Ativa'
      AND i.id_jogador NOT IN (SELECT id_jogador FROM Presenca WHERE
          id_treino = @id_treino);
END;
GO
```

sp_LiquidarMensalidade

```
CREATE OR ALTER PROCEDURE sp_LiquidarMensalidade
    @id_jogador      INT,
    @id_escalao_epoca INT,
    @metodo_pagamento VARCHAR(30)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @id_inscricao INT, @id_quota INT, @valor DECIMAL(5,2);
    SELECT @id_inscricao = id_inscricao FROM Inscricao
```

```
    WHERE id_jogador = @id_jogador AND id_escalao_epoca =
        @id_escalao_epoca
      AND estado = 'Ativa';
    IF @id_inscricao IS NULL THROW 53001, 'Inscricao ATIVA nao
        encontrada.', 1;
    SELECT TOP 1 @id_quota = id_quota, @valor = valor FROM Quota
    WHERE id_inscricao = @id_inscricao AND pago = 0 ORDER BY
        data_vencimento ASC;
    IF @id_quota IS NULL THROW 53002, 'Sem quotas por pagar.', 1;
    INSERT INTO Pagamento (data_pagamento, valor_pago, metodo_pagamento,
        id_quota)
    VALUES (CAST(GETDATE() AS DATE), @valor, @metodo_pagamento,
        @id_quota);
    UPDATE Quota SET pago = 1 WHERE id_quota = @id_quota;
END;
GO
```